

Jahrmarkt

Nayra ist auf einem Jahrmarkt und spielt ein Spiel, bei dem der Hauptpreis eine Reise nach Laguna Colorada ist. Das Spiel besteht daraus, mit Token Lose zu kaufen. Durch den Kauf eines Loses kann man zusätzliche Token bekommen. Das Ziel ist es, so viele Lose wie möglich zu kaufen.

Sie beginnt das Spiel mit A Token. Es gibt N Lose, nummeriert von 0 bis $N - 1$. Nayra muss $P[i]$ Token ($0 \leq i < N$) zahlen, um das Los i zu kaufen (und sie muss vor dem Kauf mindestens $P[i]$ Token haben). Sie kann jedes Los höchstens einmal kaufen.

Darüber hinaus wird jedem Los i ($0 \leq i < N$) ein **Typ** zugewiesen, bezeichnet mit $T[i]$. Ein Typ kann **1**, **2**, **3** oder **4** sein. Nachdem Nayra das Los i gekauft hat, wird die Anzahl ihrer verbleibenden Token mit $T[i]$ multipliziert. Formal gilt: Wenn sie zu einem bestimmten Zeitpunkt des Spiels X Token hat, und sie kauft Los i (was $X \geq P[i]$ erfordert), dann hat sie nach dem Kauf $(X - P[i]) \cdot T[i]$ Token.

Deine Aufgabe ist es, zu bestimmen, welche Lose in welcher Reihenfolge Nayra kaufen soll, um die Anzahl an insgesamt gekauften **Losen** zu maximieren. Wenn dies durch unterschiedliche Kaufsequenzen erreicht werden kann, kannst du eine beliebige dieser bestimmen.

Details zur Implementierung

Implementiere folgende Funktion:

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : die initiale Anzahl an Token, die Nayra hat.
- P : eine Liste der Länge N , die die Preise der Lose angibt.
- T : eine Liste der Länge N , die die Typen der Lose angibt.
- Diese Funktion wird für jeden Testfall genau einmal aufgerufen.

Die Funktion soll eine Liste R zurückgeben, die Nayras Einkäufe wie folgt angibt:

- Die Länge von R soll der maximalen Anzahl an Losen entsprechen, die sie kaufen kann.
- Die Elemente der Liste sind die Indizes der Lose, die sie kaufen sollte, in chronologischer Reihenfolge. Das heißt, sie kauft zuerst das Los $R[0]$, dann das Los $R[1]$ und so weiter.
- Alle Elemente von R müssen unterschiedlich sein.

Wenn keine Lose gekauft werden können, sollte R eine leere Liste sein.

Beschränkungen

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ für alle i mit $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ für alle i mit $0 \leq i < N$.

Teilaufgaben

Teilaufgabe	Punkte	Weitere Beschränkungen
1	5	$T[i] = 1$ für alle i mit $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ für alle i mit $0 \leq i < N$.
3	12	$T[i] \leq 2$ für alle i mit $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra kann alle N Lose kaufen (in irgendeiner Reihenfolge).
6	16	$(A - P[i]) \cdot T[i] < A$ für alle i mit $0 \leq i < N$.
7	18	Keine weiteren Beschränkungen.

Beispiele

Beispiel 1

Betrachte folgenden Funktionsaufruf:

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra hat zunächst $A = 13$ Token. Sie kann 3 Lose in der unten gezeigten Reihenfolge kaufen:

Gekauftes Los	Lospreis	Lostyp	Anzahl der Token nach dem Kauf
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

In diesem Beispiel ist es Nayra nicht möglich, mehr als 3 Lose zu kaufen. Die oben beschriebene Kaufreihenfolge ist die einzige Möglichkeit, wie sie 3 Lose kaufen kann. Daher sollte die Funktion $[2, 3, 0]$ zurückgeben.

Beispiel 2

Betrachte folgenden Funktionsaufruf:

```
max_coupons(9, [6, 5], [2, 3])
```

In diesem Beispiel kann Nayra beide Lose in beliebiger Reihenfolge kaufen. Daher sollte die Funktion entweder $[0, 1]$ oder $[1, 0]$ zurückgeben.

Beispiel 3

Betrachte folgenden Funktionsaufruf:

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

In diesem Beispiel hat Nayra einen Token, der nicht ausreicht, um Lose zu kaufen. Daher sollte die Funktion $[]$ (ein leere Liste) zurückgeben.

Beispiel-Grader

Eingabeformat:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Ausgabeformat:

```
S
R[0] R[1] ... R[S-1]
```

Hier ist S die Länge der Liste R , die von `max_coupons` zurückgegeben wird.